SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I S.HARISH/192521003 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1

A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

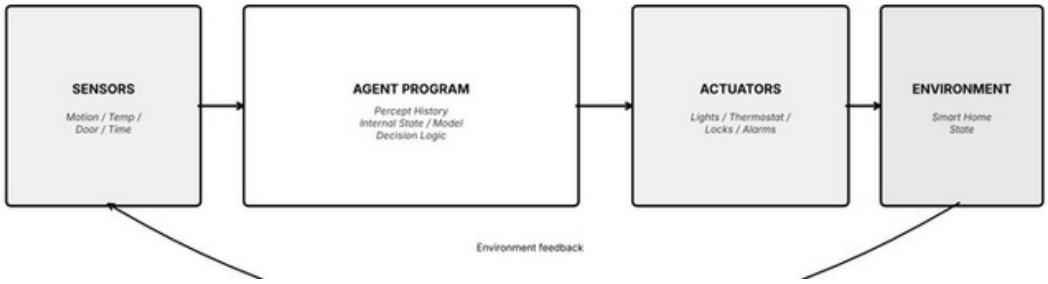
Answer

Introduction

An intelligent agent is any entity that perceives its environment through sensors and acts upon that environment through actuators. The agent receives a sequence of percepts from the environment and produces actions in response. The collection of all percepts received so far is called the percept history. An agent program maps each possible percept sequence to an action, while an agent function mathematically describes the agent's behavior.

In a smart home system, sensors include motion detectors, temperature sensors, door/window sensors, and real-time clocks. Actuators include lighting circuits, thermostats, door locks, alarms, and appliances. The agent program processes sensor input and decides which actuators to activate.

Figure 1: Smart Home Intelligent Agent Architecture



Simple Reflex Agent

Definition and WorkingPrinciple

A simple reflex agent selects actions based solely on the current percept, ignoring all previous percepts. Its decision logic consists entirely of condition-action rules of the form: IF condition THEN action. It does not maintain any memory of past states and does not model the world; it simply matches the current percept to a rule and fires the corresponding action.

Model-Based Reflex Agent

Definition and Working Principle

A model-based reflex agent overcomes partial observability by maintaining an internal state that summarizes what is currently happening in the environment. It uses two types of knowledge: a transition model (how the world evolves over time) and a sensor model (how the world state is reflected in percepts). By combining the internal state with the current percept, the agent can form a more complete picture of the environment before selecting an action.

Goal-Based Agent

Definition and Working Principle

A goal-based agent acts in order to achieve a specified goal or set of goals. Rather than simply reacting to the current percept or internal state, it considers which actions will lead to a state in which the goal is satisfied. This requires some form of search or planning: the agent must look ahead and evaluate sequences of actions in order to find one that achieves the goal.

Utility-Based Agent

Definitionand Working Principle

A utility-based agent selects actions by evaluating a utility function, which assigns a numeric score to each possible state of the world. Higher utility corresponds to a more desirable state. Rather than merely asking whether a goal has been achieved, the agent asks how desirable an outcome is. This allows it to make rational decisions even when goals conflict and to choose the best available action under uncertainty.

Comparison of Agent Types

Agent Type	Decision Method	Uses Memory / State	Advantages	Limitations	Smart Home Application
Simple Reflex	Condition-action rules on current percept	No	Simple, fast, low cost	Cannot handle history; fails in partial observability	Emergency alarms, basic lighting on/off
Model-Based Reflex	Condition-action rules with internal state tracking	Yes (internal state)	Handles partial observability; tracks room occupancy	Model may become outdated; still rule-based	Occupancy tracking, avoiding redundant actions
Goal-Based	Plans actions to reach a desired goal state	Yes (goals + state)	Flexible; can plan sequences; goal-driven	Does not optimise between conflicting goals	Maintaining target temperature; locking doors
Utility-Based	Maximises expected utility function	Yes (state + utility model)	Balances competing goals; rational under uncertainty	Utility function design is complex	Balancing comfort, energy saving, and security

Practical Scenario:Night-Time Operation

Consider the following situation: it is 23:30; the living room has been unoccupied for 30 minutes; a motion sensor suddenly detects movement in the hallway; the outdoor temperature is 32°C; and the indoor temperature is 27°C.

The following is a step-by-step account of how the system responds:

- Step 1 — Sensor input: The motion sensor in the hallway sends a signal. The temperature sensor reads 27°C indoors. The time module reports 23:30. The living room occupancy sensor reports no movement for 30 minutes.
- Step 2 — State update: The agent updates its internal state. It records that hallway motion has been detected at 23:30. It retains the knowledge that the living room has been unoccupied.

- Step 3 — Security evaluation: Since it is past 22:00 and motion is detected in theha llway, the security sub-system evaluates this as a potential intrusion event. A camera is activated in the hallway for verification.

- Step 4 — Lighting decision: The hallway light is switched on (it is dark and motion has been detected). The living room light remains off, since the room is unoccupied. This avoids unnecessary energy use.

- Step 5 — Temperature decision: The utility function evaluates whether to run the air conditioner. Since the room is unoccupied and the time is late, it selects a ceiling fan rather than the air conditioner to maintain a comfortable ambient temperature at low energy cost.

- Step 6 — Output: Hallway light on; camera active; living room light off; ceiling fan on; air conditioner off. An alert is sent to the occupant's mobile device to confirm the motion event.

Question 2

A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Answer

Uninformed Search

Uninformed search strategies, also called blind search strategies, explore the search space without using any domain-specific knowledge about the location of the goal. They do not possess a heuristic estimate of the distance remaining to the goal; they rely only on information available in the problem definition: the current state, the available actions, the transition model, the path cost, and the goal test. Both Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) are uninformed strategies. They are called uninformed because they treat all unexplored states equally, without guiding the search towards the goal using a heuristic function such as estimated straight-line distance.

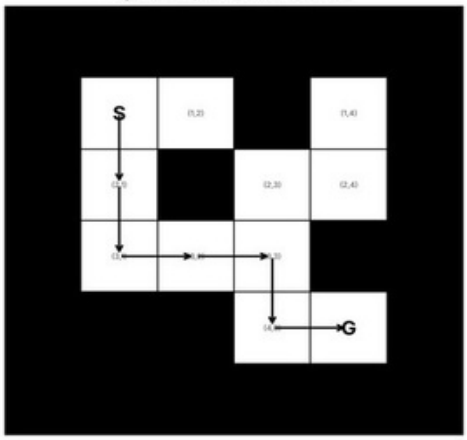
Th e Maze as a Search Problem

The robot's maze-navigation problem can be formally stated as follows:

- Initial state: The robot's starting cell, denoted S.
- State space: The set of all cells in the maze that the robot can reach from the start by legal moves.
- Actions: Move up, move down, move left, or move right. An action is legal only if it leads to a non-wall cell within the maze boundary.
- Transition model: RESULT(state, action) = the cell reached after taking that action from the current cell.
- Goal test: Determines whether the robot has reached the exit cell, denoted G.
- Path cost: The cumulative cost of all moves taken from the start to the current cell. If all moves cost 1, the path cost equals the number of moves.

An important constraint in this problem is that the robot starts in an unknown maze. It does not have access to a complete map at the beginning. It must discover passable paths by moving and using sensors, update its internal representation of the maze, and replan as new information becomes

Figure 2: Illustrative Maze — S=Start, G=Goal, #=Wall
Optimal Path (cost=6) shown with arrows



The reachable open cells and their coordinates are: S=(1,1), (1,2), (1,4), (2,1), (2,3), (2,4), (3,1), (3,2), (3,3), (4,3), G=(4,4). Adjacency (ignoring walls): (1,1)↔(1,2), (1,1)↔(2,1), (1,2) is blocked below by wall at (2,2), (2,1)↔(3,1), (3,1)↔(3,2), (3,2)↔(3,3), (3,3)↔(2,3), (2,3)↔(2,4), (2,3)↔(1,3) blocked (#), (3,3)↔(4,3), (4,3)↔(4,4=G). Also (1,2)→(1,1) back and (1,4) reachable from (2,4) up.

UCS trace (expanding lowest-cost node first; each step costs 1):

Step	Node Expanded	g(n)	Successors Added to Frontier	Priority Queue after step (node: cost)
0	—	—	S(1,1): 0	S:0
1	S=(1,1)	0	(1,2):1, (2,1):1	(1,2):1, (2,1):1 (2,1):1
2	(1,2)	1	(no new unvisited neighbours beyond (1,1))	
3	(2,1)	1		(3,1):2
4	(3,1)	2	(3,1):2	(3,2):3
5	(3,2)	3	(3,2):3	(3,3):4
6	(3,3)	4	(3,3):4 (2,3):5,	(2,3):5, (4,3):5
7	(2,3)	5	((14,,3)) :w5 all—(2,4):6 s(2k,i4p):6, G=(4,4):6	(4,3):5,
8	(4,3)	5		(2,4):6, G:6

9	(2,4)	6	(no new unvisited neighbours)	G:6
10	G=(4,4)	6	GOAL FOUND — return path	—

Optimal path: S(1,1) → (2,1) → (3,1) → (3,2) → (3,3) → (4,3) → G(4,4). Total path cost = 6 moves. This is confirmed to be optimal since UCS guarantees the minimum-cost path when all step costs are non-negative.

Completeness and Optimality

UCS is complete, meaning it will always find a solution if one exists, provided the branching factor is finite and all step costs are strictly positive (greater than zero). UCS is also optimal: it always returns the minimum-cost path to the goal under non-negative step costs, because it expands nodes in order of increasing path cost and the goal is accepted only when it is dequeued, not when it is first generated.

It erative DeepeningSearch(IDS)

Algorithm Explanation

Iterative Deepening Search combines the memory efficiency of depth-first search with the completeness of breadth-first search. It works by repeatedly executing depth-limited depth-first search (DLS) with an increasing depth limit. In each iteration, DLS explores all nodes reachable within the current depth limit and returns failure if no goal is found. The depth limit is then incremented by 1 and the search is restarted from the beginning.

The iterations proceed as follows:

- Iteration with limit 0: Visit only the start node. If it is the goal, return it. Otherwise, return failure.
- Iteration with limit 1: Visit the start node and all nodes reachable in 1 step. If the goal is found, return it.
- Iteration with limit 2: Visit all nodes reachable within 2 steps. Continue checking.
- Continue increasing the limit until the goal is found.

Within each DLS run, nodes are explored depth-first (LIFO). A visited list is maintained along the current path to prevent cycles on that path; nodes already on the current path are not re-expanded. After backtracking, a node may be visited again in a subsequent branch at the same depth limit.

IDS Trace on the Same Maze

Expansion order convention: neighbours are considered in the order Down, Right,Up,L eft.Depth limit = d.

Limit = 0: Visit S(1,1). Not goal.

Limit = 1: S(1,1) → (2,1) [not goal] backtrack → (1,2) [not goal]. No goal found.

Limit = 2: S → (2,1) → (3,1) [not goal, depth 2, backtrack] → back to (2,1), no more children at depth ≤ 2 → back to S → (1,2) → (no unvisited children in depth budget). No goal found.

Limit = 3: S → (2,1) → (3,1) → (3,2) [depth 3, not goal, backtrack]. No goal at depth 3.

Limit = 4: S → (2,1) → (3,1) → (3,2) → (3,3) [depth 4, not goal, backtrack]. No goal at depth 4. Limit = 5: S → (2,1) → (3,1) → (3,2) → (3,3) → (4,3) [depth 5, not goal] backtrack → (2,3) [depth 5, not goal]. No goal at depth 5.

Limit = 6: S → (2,1) → (3,1) → (3,2) → (3,3) → (4,3) → G(4,4) [depth 6]. GOAL FOUND at depth 6. Path = S → (2,1) → (3,1) → (3,2) → (3,3) → (4,3) → G. Total moves = 6.

Advantages of IDS

- Low memory requirement: IDS uses space proportional to O(bd), where b is the branching factor and d is the current depth limit. This is much less than BFS or UCS, which must store the entire frontier.
- Complete: IDS will find a solution if one exists at any finite depth.
- Finds the shallowest solution first, since it tries all depth limits in order.

Disadvantages of IDS

- Repeated expansion of upper-level nodes: In each new iteration, all nodes at depths 1, 2, ..., d-1 are re-expanded. This introduces redundant work.
- Not optimal when step costs differ: IDS finds the shallowest path, not the minimum-cost path. If some moves cost more than others, IDS may return a longer but cheaper path or a shorter but more expensive path.
- Not suitable for graphs with large branching factors when the goal is very deep, since the redundant re-expansion becomes very costly.

T ime and Space Complexity Comparison

The following symbols are used throughout this section:

- b = branching factor (maximum number of successors of any node).
- d = depth of the shallowest goal.
- m = maximum depth of the search tree (may be infinite).
- C* = cost of an optimal solution.
- ε (epsilon) = a positive lower bound on the cost of each step (UCS assumption: all step costs ≥ ε > 0).

Criterion	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Time Complexity	O(b^(1 + floor(C*/epsilon))) Exponential in C*/epsilon under standard assumptions	O(b^d) Exponential in the depth of the shallowest goal
Space Complexity	O(b^(1 + floor(C*/epsilon))) Must store entire frontier in the priority queue	O(b*d) Only stores the current path (linear in depth)
Completeness	Yes, if b is finite and all step costs > 0	Yes, if b is finite and depth is finite
Optimality	Yes, finds minimum-cost path (non-negative step costs)	Only if all step costs are equal; finds shallowest path
Memory Use	High: stores all frontier nodes	Very low: stores only the current path
Sensitivity to Step Costs	Handles varied step costs optimally	Optimal only for uniform step costs
Suited to Maze (uniform cost)	Yes, guaranteed optimal	Yes, finds shortest path (same as optimal for uniform cost)
Suited to Maze (varied cost)	Yes, guaranteed optimal	Not guaranteed; may miss the minimum-cost path